

Formalization Seminar: Logic and Sets

George McNinch

Tufts University

2026-09-22

Propositions	1
Representing propositions in a computer language	2
Naive overview of types	3
Naive overview of the type Prop	4
Two different kinds of things	5
Implication	6
Modus Ponens	7
Conjunction	8
Disjunction	9
Negation	10
Absurd	11
Demo examples	12

- last time: we saw that Lean **checked** a proof, and it felt like a compilation step
- but what, exactly, is it checking?
- some statements a computer really can just settle by calculation
 $2 + 3 = 5$ a finite check
- most of what we actually care about can't be settled that way: “there are infinitely many primes,” “this sequence converges,” “these two groups are isomorphic” – there's no brute-force computation that decides these
- the solution is what is known as *Type Theory*

Naive overview of types

- recall from “computer science” that can represents different sorts of data with types

for example, we have natural numbers

$5 : \mathbb{N}$, rational numbers $1/2 : \mathbb{Q}$, strings `"foo" : String`

Lists of rational numbers: `[ 1, 1/2 ] : List  $\mathbb{Q}$`

Functions $u : \mathbb{N} \rightarrow \mathbb{R}$ (“sequences” in this case)

Boolean value `true : Bool` and `false : Bool`

- here is the pattern: for a type t , we can consider terms x of type t , and the notation $x : t$ means that x is a term of type t .

Naive overview of the type Prop

- We want to consider *propositions* and be able to reflect their correctness computationally.
- The idea: introduce a **type** Prop of Propositions
- If $a : \text{Prop}$ then a term $t : a$ is a *proof* of a or a *certificate* of a .
- we saw this in last time's demo: `append_length` wasn't checked by running anything – it was checked by verifying the **proof term** has the right type and the **proof term** is obtained by coding it in Lean (either directly, or using *tactics*)

Two different kinds of things

```
#eval (2 + 2 == 5)          -- Bool: computes, gives `false`  
#check (2 + 2 = 5 : Prop)   -- Prop: a statement
```

- Bool is ordinary **data** – two constructors, true and false. You **compute** a Bool.
- Prop is the type of **statements**. To settle one you give a **proof**

```
example : ¬ (2 + 2 = 5) := by decide
```

- the tactic decide actually produces a proof term $t : \neg(2+2 = 5)$

decide can settle statements like this automatically, because they're the kind of statement a computer can just check by calculation.

- but! There are lots of propositions which aren't Decidable.

e.g. for $a : \mathbb{R}$, the proposition $a = 0 : \text{Prop}$ is not decidable.

Indeed, to Lean, a real number is represented by a Cauchy sequence $\mathbb{N} \rightarrow \mathbb{Q}$

Implication

- Given $p \ q : \text{Prop}$, the type of functions $p \rightarrow q$ is again a Prop :

$p \rightarrow q : \text{Prop}$

- Let's think through why:

a term $f : p \rightarrow q$ should be thought of as a function from “proofs of p ” to “proofs of q ”

- so in fact $p \rightarrow q$ can be read as “ p implies q ”

Here is how to state and prove *modus ponens* in Lean:

```
theorem modus_ponens {p q : Prop} : (p → q) → p → q :=  
  fun f h => f h
```

Or slightly more readably

```
theorem modus_ponens' {p q : Prop} : (p → q) → p → q := by  
  intro f h -- now `f : p → q` and `h : p`  
  exact f h
```

Conjunction

Given $p \ q : \text{Prop}$, we get a proposition $p \wedge q : \text{Prop}$ which should be true precisely when both p and q are true.

Mechanically, this is achieved in Lean by

```
example {p q : Prop} (hp : p) (hq : q) : p ∧ q :=  
  And.intro hp hq
```

In practice, often `And.intro` is used under the hood by a tactic.

```
example {p q : Prop} (hp : p) (hq : q) : p ∧ q := by  
  constructor -- this tactic produces *two* goals  
  · exact hp  
  · exact hq
```

Disjunction

Given $p \ q : \text{Prop}$, we get a proposition $p \vee q : \text{Prop}$ which should be true precisely when either p or q is true.

We need more *introduction rules* to handle disjunction.

```
variable (p q : Prop)
```

```
example (hp : p) : p ∨ q := Or.intro_left q hp
```

```
example (hq : q) : p ∨ q := Or.intro_right p hq
```

Negation

- in contrast to the Bool values true and false there are *types*

True : Prop and False : Prop.

The type True is a “trivially provable” proposition. In Lean, its proof term (certificate) is called

True.intro : True

The type False has no terms.

- $\neg P$ is **notation** for $P \rightarrow \text{False}$
- so “prove $\neg P$ ” means: construct a function taking a hypothetical proof of P and producing a proof of False

Note that False implies **anything**

```
#check False.elim
```

```
-- False.elim.{u} {C : Sort u} (h : False) : C
```

```
#check absurd
```

```
-- absurd.{v} {a : Prop} {b : Sort v} (h1 : a) (h2 : ¬a) : b
```

```
example (a b : ℝ) (h : a = b) (k : a ≠ b) : 1 = 0 :=  
  absurd h k
```

```
import Mathlib
/-! demo -/
```

```
example {x y : ℝ} (h : x ≤ y ∧ x ≠ y) : ¬y ≤ x := by
  rcases h with ⟨h₀, h₁⟩
  contrapose! h₁
  exact le_antisymm h₀ h₁
```

```
example {m n k : ℕ} (h : m | n ∨ m | k) : m | n * k := by
  rcases h with ⟨a, rfl⟩ | ⟨b, rfl⟩
  · rw [mul_assoc]
    apply dvd_mul_right
  · rw [mul_comm, mul_assoc]
    apply dvd_mul_right
```